

Musterlösungen

01 | b) Eine Klasse, die ausschließlich das Datum ohne Uhrzeit speichert.

`LocalDate` in Java speichert nur das Datum (Jahr, Monat, Tag), ohne Informationen zur Uhrzeit oder Zeitzone.

02 | d) Alle oben genannten Methoden.

`LocalDate` bietet Methoden wie `now()`, um das heutige Datum zu erhalten, `toString()`, um das Datum als String zu formatieren, und `isLeapYear()`, um zu prüfen, ob es ein Schaltjahr ist.

03 | a) `LocalDate.now()`

Mit `LocalDate.now()` können Sie das aktuelle Datum in Form eines `LocalDate`-Objekts erhalten.

04 | a) `LocalDate.of()`

Die Methode `LocalDate.of()` ermöglicht es Ihnen, ein `LocalDate`-Objekt mit einem bestimmten Jahr, Monat und Tag zu erstellen.

05 | a) `localDate.getYear()`

Um das Jahr eines `LocalDate`-Objekts zu extrahieren, verwenden Sie die Methode `getYear()`.

06 | b) `plusDays(int days)`

`plusDays(int days)` wird verwendet, um ein `LocalDate` um eine bestimmte Anzahl von Tagen zu ändern.

07 | a) `localDate.isLeapYear()`

Die Methode `isLeapYear()` prüft, ob das Jahr des `LocalDate`-Objekts ein Schaltjahr ist.

08 | b) Ein `LocalDate`-Objekt mit dem Wert 2025-03-15

`LocalDate.parse("2025-03-15")` gibt ein `LocalDate`-Objekt mit dem Datum 15. März 2025 zurück.

09 | b) Es wird ein Fehler geworfen, da der Februar nur 28 oder 29 Tage hat.

Ein Versuch, ein Datum mit dem 30. Februar zu erstellen, führt zu einer `DateTimeException`, da dieser Tag im Februar nicht existiert.

10 | d) Alle oben genannten Optionen sind korrekt.

Sie können `LocalDate.compareTo(otherDate)`, `LocalDate.equals(otherDate)` oder `LocalDate.isBefore(otherDate)` verwenden, um `LocalDate`-Objekte miteinander zu vergleichen.
